UNIVERSIDAD DIEGO PORTALES

FACULTAD DE INGENIERÍA

Escuela de Ingeniería en Informática y Telecomunicaciones



Tarea N.º 02

Torneo de Komikan Concurrente

Asignatura: Sistemas Operativos
Semestre: Segundo semestre 2026

Estudiantes:

Diego Briones
Matías Rojas Lara

Santiago de Chile, 03 de noviembre de 2026

1. Diseño del esquema de sincronización

Se eligió desarrollar la tarea dentro de un solo programa que trabaja con muchos hilos, es decir, varias tareas que se ejecutan de manera simultánea. La razón principal de esto es que los hilos comparten la misma memoria. Otra opción era usar un programa distinto para cada ficha, pero habría que crear memoria compartida del sistema operativo, pero si el programa se corta de golpe con Ctrl+C podrían quedar "abandonadas" dentro del computador. Con hilos esto no sucede, ya que todo lo que se crea se libera al término.

Se utilizó un hilo para cada cliente, porque la tarea pide atender a N clientes a la vez, un hilo para cada ficha (1 puma y 12 perros), ya que así cada una decide por su cuenta, con su propio tiempo al azar y la competencia por las casillas es real y no una simulación. Cada mesa tiene además su propio hilo Juez, como es solicitado en el enunciado. Un hilo se encarga de juntar a los jugadores y otro hilo vigila constantemente el Ctrl+C para finalizar las partidas.

Para evitar colisiones se usó un juez único por mesa en vez de un mutex por casilla. Las fichas no escriben sobre el tablero, encolan una solicitud y esperan la respuesta del juez, que las aplica de a una. El estado de la mesa se protege con un mutex y dos variables de condición, una despierta al juez cuando llega una solicitud y la otra despierta a las fichas cuando ya fueron atendidas. Así solo hay un punto donde cambia el estado y la exclusión mutua queda garantizada porque ningún otro hilo modifica el tablero, lo que facilita que no ocurran inconsistencias. Una captura toca tres casillas. Si el puma está acorralado, obliga a mirar varias a la vez, por lo que con un mutex por casilla habría que tomar muchos locks en un orden global, lo que complejiza el código y podrían ocurrir deadlocks. Las solicitudes entran a una cola circular FIFO, ya que así las colisiones se resuelven como es solicitado en el enunciado (la primera ficha que reclame una casilla se la queda). Como una ficha decide mirando un tablero el cual puede cambiar antes de que el juez la atienda, este siempre revalida que la ficha esté en el origen, que el destino esté libre y que el paso o salto sea válido. Por eso, de dos fichas que pidieron la misma casilla, la segunda recibe "casilla ocupada".

Para que nunca haya más de M mesas jugando se utilizó un semáforo POSIX iniciado en M, ya que solo permite esa cantidad de cupos. Antes de abrir una mesa, el matchmaker (organizador) pide un cupo. Si no hay, se queda esperando sin consumir CPU, es decir, "espera pasiva", como es solicitado en el enunciado. Cuando una mesa termina, devuelve su cupo y el organizador continúa con la siguiente pareja. Las colas de espera de Pumas y de Perros se protegen con un mutex y una variable de condición del servidor, ya que el matchmaker debe dormir hasta que existan ambos roles y los clientes deben avisarle cuando llegan.

Para la caída de un jugador se usó un mutex robusto (PTHREAD_MUTEX_ROBUST) en cada mesa, ya que el enunciado pide desbloquear lo que retenga el jugador que falló. Con un mutex normal, si un hilo muere reteniéndolo, la mesa quedaría bloqueada para siempre. Con uno robusto, el siguiente hilo recibe EOWNERDEAD, lo deja consistente y el juez declara ganador al oponente por abandono. Si el usuario presiona Ctrl+C, el servidor deja de aceptar jugadores. La señal se bloquea en todos los hilos y la espera un único hilo con sigtimedwait, en lugar de usar un manejador asíncrono, que solo podría ejecutar operaciones muy limitadas. Ese hilo avisa a cada mesa activa para que termine ordenadamente. Luego se hace join de todos los hilos, se destruyen los mutex, las variables de condición, el semáforo y se cierra el archivo de registro.

2. Análisis de riesgos de concurrencia

Durante el desarrollo de esta tarea se tuvo en cuenta los siguientes riesgos de concurrencia, deadlock, livelock e inanición.

Para evitar deadlock se hicieron tres cosas. La primera, se creó un orden fijo para pedir llaves (primero la del servidor, después la de la mesa, después la del registro) que se respeta siempre. En segundo lugar, el tablero usa una sola llave (mutex), así que no se puede formar ese problema entre casillas y nadie se duerme ni espera un cupo del semáforo con una llave en la mano. En tercer lugar, las fichas esperan su tiempo al azar antes de pedir la llave, no mientras la tienen.

Para evitar livelock, que aquí ocurriría si varias fichas chocaran siempre por la misma casilla, una

ficha rechazada no reintenta de inmediato, sino que espera un tiempo distinto al azar y vuelve a mirar el tablero actualizado. Así es muy poco probable que sigan chocando. También puede pasar que una partida no termine nunca (los Perros no logran acorralar al Puma y el Puma no logra capturar), por lo que se puso un tope de 4000 jugadas que termina la partida en empate. Es solo una protección. Las partidas normales terminan mucho antes.

La inanición ocurriría si una ficha nunca fuera atendida. Como el juez atiende en orden FIFO y cada ficha tiene a lo más una solicitud pendiente, en el peor caso espera a las otras 12. Las colas de emparejamiento son FIFO por la misma razón. El semáforo solo lo espera un hilo (el matchmaker), así que no compite con nadie.

3. Evaluación de las decisiones de diseño

Para el desarrollo de esta tarea se consideró la complejidad de implementación, el rendimiento y el nivel de concurrencia. En los tres casos se priorizó una solución simple y fácil de verificar por sobre el máximo paralelismo, ya que las condiciones reales del juego no exigían más. Cada mesa cuenta con un juez único, que es el único que modifica el tablero. También cuenta con un solo mutex que protege su estado. De esta forma, basta con comprobar que el tablero solo cambia dentro del juez y que cada movimiento se valida antes de aplicarse. Se descartó usar un mutex por casilla porque una captura involucra tres casillas y detectar al puma acorralado exige revisar varias a la vez, lo que obligaría a tomar muchos locks en un orden global y aumentaría el código y el riesgo de deadlock.

Aunque el juez podría ser un cuello de botella, en la práctica la carga es baja. Cada ficha se mueve cada 50 a 300 ms y atender una solicitud toma mucho menos que eso, por lo que la cola casi siempre está vacía. Aun en el peor caso, como cada ficha tiene a lo más una solicitud pendiente, un movimiento espera como máximo a otras 12 solicitudes. En la prueba de estrés, con 1 ms de retardo, se procesaron 500 solicitudes en cerca de 0,5 s, de las cuales 490 fueron aceptadas y 10 rechazadas (9 por colisión de casilla). El tablero terminó consistente. Entre mesas la concurrencia es total, ya que cada una tiene su propio estado y solo comparten el semáforo de mesas y el registro. Dentro de una mesa solo se serializa la escritura del tablero.

Además, varias decisiones se tomaron pensando en seguir desarrollando el programa. El estado vive dentro de cada mesa y no en variables globales. Las casillas vecinas se calculan en una tabla, de modo que cambiar el trono o las diagonales se hace en un solo lugar. Las solicitudes tienen espacio fijo, por lo que no se pide memoria en cada movimiento. Por último, la configuración se lee por palabras clave, así que agregar un parámetro nuevo requiere solo una línea.